Proving Data Structures Correct: Balanced Search Trees

Computer-Assisted Theorem Proving (7CCSACTP)

Mohammad Abdulaziz

Where We Are in the Course

- **Covered in Weeks 1–6:**
  - **Week 1:** ITP fundamentals and the Isabelle/HOL environment.
  - **Week 2:** Mathematical induction and hypothesis generalisation.
  - **Week 3:** List recursion, structural induction, and proof automation.
  - **Week 4:** Algebraic datatypes, tree induction, binary search trees.
  - **Week 5:** AI-assisted proving (Sledgehammer, coding agents via MCP).
  - **Week 6:** Correctness of sorting, computation induction, running times.
- **Week 7:** Balanced search trees: 2-3 trees, their height, and insertion.
- **Next, Week 8:** Abstract data types, implemented by search trees.

©Mohammad Abdulaziz

Outline

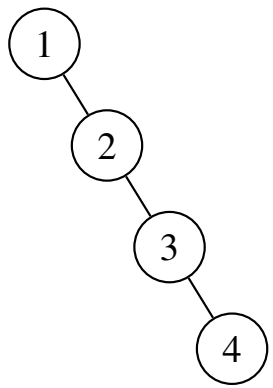
1. Balanced search trees
   - Search time depends on the height
2. 2-3 trees: data structure and search
   - The invariant `complete` and logarithmic height
3. 2-3 trees: insertion and balance
   - Overflow, and why insertion preserves `complete`

©Mohammad Abdulaziz

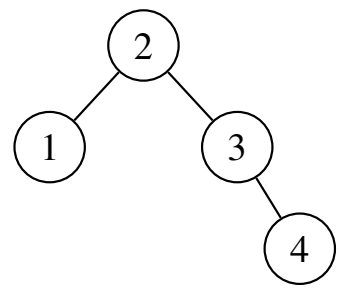
Search Time Depends on the Height

Week 4: `isin` follows one path from the root
$\Rightarrow$ search takes time $O(h(t))$, where $h(t)$ is the height of t

The height depends on the order of insertion:



`ins` 1, 2, 3, 4: height 4



`ins` 2, 1, 3, 4: height 3

Worst case: $h(t) = |t|$

Best case: $h(t) \approx \log_2 |t|$

©Mohammad Abdulaziz

Balanced Search Trees

Balanced search trees guarantee $h(t) = O(\log |t|)$

Two kinds of invariants:

- the ordering invariant, like `bst` in Week 4
- a structural invariant = a *balance criterion*

`insert` (and `delete`) must preserve both.

Examples: AVL trees, red-black trees, 2-3 trees, B-trees, ...

©Mohammad Abdulaziz